

Using Flood Fill

Graphics programs commonly use the *flood fill algorithm* to fill an arbitrarily shaped area of the same color with another color. [Figure 3-6](#) shows one such shape at the top left. The subsequent panels show three different sections of the shape flood-filled with a gray color. The flood fill begins on a white pixel and spreads until it meets a non-white pixel, filling the enclosed space.

The flood fill algorithm is recursive: it begins by changing a single pixel to a new color. The recursive function is then called on any neighbors of the pixel with its same old color. It then moves on to the neighbors of the neighbors, and so on, converting each pixel to the new color until the enclosed space is filled in.

The base case is a pixel whose color is the edge of the image or is not the old color. Since reaching the base case is the only way to stop the “spread” of recursive calls for every pixel in the image, this algorithm has the emergent behavior of changing all the contiguous pixels from the old color to the new color.

Let’s ask the three recursive algorithm questions about our `floodFill()` function:

What is the base case? When the x- and y-coordinates are for a pixel that is not the old color, or are at the edge of the image.

What arguments are passed to the recursive function call? The x- and y-coordinates of the four neighboring pixels of the current pixel are the arguments to four recursive calls.

How do these arguments become closer to the base case? The neighboring pixels run up to a different color than the old color or the edge of the image. Either way, eventually the algorithm runs out of pixels to check.